C++'a Kısa Bir Bakış: Tarihçe, Kullanım Alanları ve Çalıştırma

1. Kısa Tarihçe

C++, **1979** yılında AT&T Bell Laboratuvarları'nda **Bjarne Stroustrup** tarafından tasarlanmaya başlandı. İlk adı **"C with Classes"** (Sınıflı C) idi; amaç, C dilinin yüksek performansını korurken nesne yönelimli programlama gibi üst düzey soyutlama yetenekleri eklemekti. **1985** yılında AT&T ilk ticari sürümü yayımladı ve aynı yıl Stroustrup'un *The C++ Programming Language* kitabının ilk baskısı çıktı; böylece dil resmen **C++** adını aldı.

Sonraki yıllarda C++ birçok kez standartlaştırıldı: **1998**'de ilk ISO standardı (C++98), ardından C++03, C++11, C++14, C++17, C++20 ve en son **C++23**. Standart komitesi her üç yılda bir yeni bir sürüm yayımlar; **geriye dönük uyumluluk** ve **kararlılık** ön plandadır. Stroustrup, C++'ın başarısını iki temel tasarım hedefine bağlar: **donanımı verimli kullanmak** ve **güçlü soyutlama mekanizmaları sunmak**; ayrıca dil, gerçek kullanımdan gelen geri bildirimlerle dikkatli biçimde evrilmiştir.

---

2. C++ ile Neler Yapılabilir?

C++, **genel amaçlı bir programlama dilidir**; hem yüksek performans hem de üst düzey soyutlama sunar. Başlıca kullanım alanları:

Alan	Açıklama
İşletim sistemleri	Windows, Linux, macOS'un çekirdek bileşenleri
Tarayıcılar	Chrome, Firefox, Safari
Oyun geliştirme	3B motorlar, çok oyunculu çevrimiçi oyunlar
Gömülü sistemler ve IoT	Akıllı cihazlar, gerçek zamanlı sistemler, sürücü yazılımları
Veritabanları	Yüksek performanslı veritabanı yönetim sistemleri
Bilimsel hesaplama	Hesaplama yoğun simülasyon ve analizler
Yüksek frekanslı işlem	Finans alanında düşük gecikmeli sistemler

C++'ın öne çıkan avantajları: **programcı kontrollü bellek yönetimi**, **zengin standart kütüphane (STL)**, **C ile uyumluluk** ve **platformlar arası çalışabilirlik**.

---

3. Çalıştırılabilecek Platformlar

C++ **platformlar arası** bir dildir; aynı kaynak kodu farklı işletim sistemlerinde derlenip çalıştırılabilir. Microsoft C++ örneğinde hedeflenebilecek platformlar:

- **Masaüstü**: Windows (x86/x64/ARM64), Linux (x86/x64/ARM64)
- **Mobil**: Android, iOS

- **Sunucu**: Windows Server serisi

> Not: Windows 7/8, Windows Server 2008 R2 gibi eski sistemlerin resmî desteği yeni Visual Studio sürümlerinde kaldırılmıştır; ancak eski araç setleriyle derleme hâlâ mümkündür.

---

4. C++ Programı Nasıl Yazılır ve Çalıştırılır?

4.1 Temel Akış

C++ programının oluşturulması şu adımları izler: **düzenle → derle → bağla → çalıştır**

1. **Kaynak dosya oluşturma**: Metin editörü veya IDE ile `.cpp` dosyası oluşturulur
2. **Derleme**: Kaynak kod makine koduna çevrilir
3. **Bağlama**: Derlenen kod, kütüphanelerle birleştirilerek çalıştırılabilir dosya üretilir
4. **Çalıştırma**: Üretilen çalıştırılabilir dosya yürütülür

4.2 IDE Kullanarak (Visual Studio Örneği)

Visual Studio'da:

- **Proje oluşturma**: `Dosya → Yeni → Proje` yolundan "Konsol Uygulaması (C++)" seçilir
- **Kod yazma**: `.cpp` dosyasına kod girilir
- **Derleme**: `Derle → Çözümü Derle` (veya `Ctrl+Shift+B`)
- **Çalıştırma**: `Hata Ayıkla → Hata Ayıklamadan Başlat` (veya `Ctrl+F5`)

Çalıştırma sonrası konsol penceresi program çıktısını gösterir; kapatmak için herhangi bir tuşa basılır.

4.3 Komut Satırı Kullanarak

Terminal/komut isteminde:

```
# Derleme (GCC örneği)
g++ hello.cpp -o hello

# Çalıştırma
./hello
```

Windows'ta üretilen dosya `hello.exe` olur; doğrudan dosya adı yazılarak çalıştırılır.

4.4 En Küçük Örnek

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Merhaba Dünya!" << endl;
    return 0;
}
```

Kodun önemli noktaları:

- `#include <iostream>`: Giriş/çıkış akışı kütüphanesini dahil eder
- `using namespace std;`: Standart isim alanını etkinleştirir; her seferinde `std::` yazmayı önler
- `main()`: Her C++ programının giriş fonksiyonu
- `cout`: Standart çıkış; `<<` operatörü içeriği çıkış akışına gönderir
- `endl`: Satır sonu verir ve tamponu boşaltır
- Her ifade `;` ile biter

---

5. Temel Terimler

Terim	Anlamı
Derleyici	C++ kaynak kodunu makine koduna çeviren araç (GCC, Clang, MSVC)
IDE	Tümleşik geliştirme ortamı; düzenleme, derleme ve hata ayıklamayı bir arada sunar (Visual Studio, CLion)
STL	Standart Şablon Kütüphanesi; konteynerler (vector, map vb.) ve algoritmalar sağlar
RAII	"Kaynak Edinimi Başlatmadır"; C++'ın temel kaynak yönetimi ilkesi
İsim alanı	İsim çakışmalarını önleyen mekanizma (örn. `std`)

`#include <iostream>` ve `using namespace std;` Nedir, Neden Her Kodun Başında Yer Alır?

C++'ta neredeyse her örnek kodun en üstünde şu iki satırı görürsünüz:

```cpp
#include <iostream>
using namespace std;
```

Bu iki satır, **birbirinden farklı iki ayrı iş** yapar. Aşağıda her birini ayrı ayrı açıklayalım.

---

1. `#include <iostream>` Nedir?

1.1 Anlamı

`#include`, C++'ta bir **önişlemci komutudur** (preprocessor directive). Derleyici kodu derlemeden **önce** çalışan önişlemciye şunu söyler:

> "`iostream` adlı başlık dosyasının içeriğini, bu satırın olduğu yere kopyala."

`iostream` ise **input/output stream** (giriş/çıkış akışı) kelimelerinin kısaltmasıdır. İçinde şu temel bileşenler tanımlıdır:

Bileşen	Görevi
`cin`	Klavyeden (standart giriş) veri okumak
`cout`	Ekrana (standart çıkış) veri yazmak
`cerr`	Hata mesajlarını yazmak
`clog`	Günlük (log) mesajlarını yazmak
`endl`	Satır sonu ve tampon boşaltma

1.2 Neden Gerekli?

C++'ta `cout`, `cin`, `endl` gibi isimler **otomatik olarak hazır gelmez**. Bu isimlerin tanımlı olduğu başlık dosyası programa dahil edilmelidir. Eğer `#include <iostream>` satırı yazılmazsa:

```
#include <iostream>   // BU SATIR OLMASAYDI:
using namespace std;

int main() {
    cout << "Merhaba";   // HATA: 'cout' tanımlı değil
    return 0;
}
```

Derleyici `cout` ve `endl`'in ne olduğunu bilemez ve **"identifier not found"** (tanımlayıcı bulunamadı) hatası verir.

1.3 Neden `< >` İçinde?

`#include` iki farklı biçimde yazılabilir:

```
#include <iostream>    // Açılı parantez: sistem/standart kütüphane
başlıkları için
#include "benim_dosyam.h"  // Çift tırnak: kendi yazdığınız başlık
dosyaları için
```

- `< >` → Derleyici, başlık dosyasını **sistem kütüphane dizinlerinde** arar.

- `" "` → Derleyici, önce **proje klasöründe** arar, bulamazsa sistem dizinlerine bakar.

`iostream` standart kütüphanenin bir parçası olduğu için `< >` ile yazılır.

1.4 `#include` Bir "Kopyala-Yapıştır" İşlemidir

Önişlemci, `#include <iostream>` gördüğünde `iostream` dosyasının **tüm içeriğini** o satırın yerine kopyalar. Yani siz aslında yüzlerce satırlık bir kod yazmış gibi olursunuz. Bu yüzden `#include` satırı **derleme öncesi** gerçekleşir ve sonunda noktalı virgül (`;`) **konulmaz** — çünkü bu bir C++ ifadesi değil, önişlemci komutudur.

---

2. `using namespace std;` Nedir?

2.1 "Namespace" (İsim Alanı) Kavramı

Büyük projelerde farklı kütüphaneler aynı isimleri kullanabilir. Örneğin hem sizin yazdığınız bir `cout` fonksiyonu, hem de standart kütüphanenin `cout` nesnesi olabilir. Bunu ayırt etmek için C++ **isim alanları** (namespaces) kullanır.

Standart kütüphanedeki her şey `std` isim alanı içindedir. Yani:

- `cout` → aslında `std::cout`
- `cin` → aslında `std::cin`
- `endl` → aslında `std::endl`
- `string` → aslında `std::string`

Burada `std`, **"standard"** kelimesinin kısaltmasıdır.

2.2 `using namespace std;` Ne Yapar?

Bu satır, derleyiciye şunu söyler:

> "Bu dosyada `std::` önekini yazmadan da `std` isim alanındaki isimlere erişebilirsin."

Yani:

```
using namespace std;

cout << "Merhaba" << endl;
```

yazmak, şuna eşdeğerdir:

```
std::cout << "Merhaba" << std::endl;
```

2.3 Neden Her Kodda Yer Alır?

Çünkü başlangıç seviyesindeki kodlarda sürekli `std::cout`, `std::cin`, `std::endl` yazmak **okunabilirliği azaltır** ve **yazım yükünü artırır**. Eğitim amaçlı örneklerde bu satır, kodu sadeleştirmek için neredeyse standart hâle gelmiştir.

Örnek karşılaştırma:

```
// using namespace std; OLMADAN
#include <iostream>
int main() {
    std::cout << "Merhaba" << std::endl;
    int x;
    std::cin >> x;
    return 0;
}
```

```
// using namespace std; İLE
#include <iostream>
using namespace std;
int main() {
    cout << "Merhaba" << endl;
    int x;
    cin >> x;
    return 0;
}
```

İkisi de aynı işi yapar; ikincisi daha kısa ve okunması daha kolaydır.

2.4 Alternatif Kullanımlar

`using namespace std;` yerine daha **seçici** yöntemler de vardır:

```
// Yalnızca belirli isimleri kullanıma aç
using std::cout;
using std::cin;
using std::endl;
```

veya hiç açmadan doğrudan `std::` önekiyle yazmak:

```
std::cout << "Merhaba" << std::endl;
```

2.5 Uyarı: Büyük Projelerde `using namespace std;` Önerilmez

Küçük örneklerde sorun çıkarmasa da, büyük projelerde `using namespace std;` **isim çakışmalarına** yol açabilir. Örneğin kendi yazdığınız bir `count` fonksiyonu, `std::count` algoritmasıyla çakışabilir. Bu

yüzden profesyonel kodda genellikle:

- Ya `std::` öneki açıkça yazılır,
- Ya da yalnızca ihtiyaç duyulan isimler `using std::cout;` gibi tek tek açılır.

Ancak **ders notlarında ve eğitim örneklerinde** `using namespace std;` yaygın ve kabul edilebilir bir kullanımdır.

---

3. İki Satırın Birlikte Özeti

```
#include <iostream>      // 1. cout, cin, endl gibi araçların TANIMLARINI getirir
using namespace std;     // 2. Bu araçlara std:: yazmadan ERİŞİM sağlar
```

Satır	Ne yapar?	Olmazsa ne olur?
`#include <iostream>`	Giriş/çıkış kütüphanesini programa dahil eder	`cout`, `cin`, `endl` tanınmaz, derleme hatası
`using namespace std;`	`std::` önekini yazma zorunluluğunu kaldırır	Kod çalışır ama her yerde `std::cout`, `std::cin` yazmak gerekir

Benzetme:

- `#include <iostream>` → Mutfağa malzemeleri getirmek (un, şeker, yumurta)
- `using namespace std;` → Malzemeleri isimleriyle çağırmak (her seferinde "dolaptaki un" demek yerine sadece "un" demek)

---

4. Sık Sorulan Sorular

Soru: `#include <iostream>` yerine `#include <iostream.h>` yazsam olur mu? **Cevap:** Hayır. `.h` uzantılı biçim, C++'ın standardizasyonundan **önceki** eski kullanımdır. Modern C++ standart kütüphane başlıkları uzantısız olur (`iostream`, `string`, `vector` vb.). Eski `.h` biçimi hâlâ bazı derleyicilerde çalışsa da **kullanılmamalıdır**.

Soru: `using namespace std;` satırını `main()` içine yazsam olur mu? **Cevap:** Olur, ama genellikle dosyanın en üstüne, `#include` satırlarının hemen altına yazılır. Böylece dosyadaki tüm fonksiyonlar bundan yararlanır. `main()` içine yazarsanız yalnızca `main()` içinde geçerli olur.

Soru: Her `.cpp` dosyasına bu iki satırı yazmak zorunda mıyım? **Cevap:** `cout`, `cin` gibi standart giriş/çıkış araçlarını kullanacaksanız evet. Eğer programınız hiç giriş/çıkış yapmıyorsa (örneğin yalnızca matematiksel hesaplama yapıp sonucu bir dosyaya yazıyorsa) `#include <iostream>` gerekmez; ama pratikte neredeyse tüm başlangıç programları bu satırı içerir.

Soru: `using namespace std;` yazmak yerine `std::` yazmak neden daha güvenli? **Cevap:** Çünkü `using namespace std;` standart kütüphanedeki **binlerce ismi** görünür hâle getirir. Bu isimlerden biri,

sizin tanımladığınız bir isimle çakışırsa derleyici hangisini kastettiğinizi bilemez. `std::` öneki bu belirsizliği ortadan kaldırır.

---

5. Özet

- `#include <iostream>` → Standart giriş/çıkış kütüphanesini programa **dahil eder**; `cout`, `cin`, `endl` gibi araçların tanımlarını getirir.
- `using namespace std;` → Standart isim alanındaki isimlere `std::` öneki olmadan **erişim sağlar**; kodu kısaltır ve okunabilirliği artırır.
- İkisi birlikte, C++'ta en temel giriş/çıkış işlemlerini yazmanın **kısa yolunu** oluşturur.
- Eğitim kodlarında yaygın ve uygundur; büyük profesyonel projelerde `using namespace std;` yerine `std::` öneki veya seçici `using` bildirimleri tercih edilir.